

A Federated Learning-Based Industrial Health Prognostics for Heterogeneous Edge Devices Using Matched Feature Extraction

Anushiya Arunan¹, Yan Qin², *Member, IEEE*, Xiaoli Li³, *Senior Member, IEEE*,
and Chau Yuen⁴, *Fellow, IEEE*

Abstract—Data-driven industrial health prognostics require rich training data to develop accurate and reliable predictive models. However, stringent data privacy laws and the abundance of edge industrial data necessitate decentralized data utilization. Thus, the industrial health prognostics field is well suited to significantly benefit from federated learning (FL), a decentralized and privacy-preserving learning technique. However, FL-based health prognostics tasks have hardly been investigated due to the complexities of meaningfully aggregating model parameters trained from heterogeneous data to form a high performing federated model. Specifically, data heterogeneity among edge devices, stemming from dissimilar degradation mechanisms and unequal dataset sizes, poses a critical statistical challenge for developing accurate federated models. We propose a pioneering FL-based health prognostic model with a feature similarity-matched parameter aggregation algorithm to discriminatively learn from heterogeneous edge data. The algorithm searches across the heterogeneous locally trained models and matches neurons with probabilistically similar feature extraction functions first, before selectively averaging them to form the federated model parameters. As the algorithm only averages similar neurons, as opposed to conventional naive averaging of coordinate-wise neurons, the distinct feature extractors of local models are carried over with less dilution to the resultant federated model. Using both cyclic degradation data of Li-ion batteries and non-cyclic data of turbofan engines, we demonstrate that the proposed method yields accuracy improvements as high

as 44.5% and 39.3% for state-of-health estimation and remaining useful life estimation, respectively. *Note to Practitioners*—Data-driven machine health monitoring enabled by cyber-physical systems allows humans to make timely predictive maintenance decisions for optimizing the life cycle of critical industrial assets. However, the benefits of intelligent health prognostics may not be fully realized in practice because strict data privacy laws restrict the aggregation of decentralized industrial data that are essential for model training. In proposing a privacy-preserving learning technique, we also focused on tackling a common and practical problem of learning from heterogeneous degradation data at the edge. The competitive performance of the proposed similarity-matching-based federated learning algorithm indicates its suitability for modeling heterogeneous, industrial time series data. Therefore, industry practitioners can utilize this algorithm as a staple component of their machine health modeling toolkit in an increasingly privacy-concerned era.

Index Terms—Federated learning, similarity matching, long short-term memory network, human-cyber-physical systems, industrial health prognostics.

I. INTRODUCTION

EDGE devices with powerful sensing and computing capabilities have grown from being a novelty to a utility, permeating all aspects of the modern industry [1]. By bringing the benefits of artificial intelligence and the Internet of Things closer to the end user for real-time analytics and decision making, edge devices play a critical role in Industry 4.0. Thus, proactive health management of these assets, through condition monitoring and timely predictive maintenance, is vital to prevent costly failures and ensure reliable operations. Life cycle costs of operating edge assets are also reduced with condition-based maintenance as a cost-effective balance between unplanned reactive maintenance and excessive preventive maintenance can be achieved [2].

Consequently, developing industrial health prognostic models to guide condition-based maintenance has attracted significant scientific and industry interest, with a growing emphasis on data-driven models that can leverage the real-time data generated by edge devices [3], [4], [5], [6]. Amongst data-driven methods, deep learning models have gained notable prominence due to their ability build accurate models from raw input data without needing extensive domain knowledge [7], [8].

Manuscript received 6 April 2023; accepted 3 May 2023. This article was recommended for publication by Editor M. Zhou upon evaluation of the reviewers' comments. This work was supported in part by the National Research Foundation, Singapore, under its AI Singapore (AISG) Program under Award AISG2-RP-2021-027. The work of Anushiya Arunan was supported by the Agency for Science, Technology and Research (A*STAR) Graduate Scholarship. (Corresponding authors: Yan Qin; Chau Yuen.)

Anushiya Arunan is with the Engineering Product Development Pillar, Singapore University of Technology and Design, Singapore 487372, and also with the Institute for Infocomm Research and the Centre for Frontier AI Research, Agency for Science, Technology and Research (A*STAR), Singapore 138632 (e-mail: anushiya_arunan@mymail.sutd.edu.sg).

Yan Qin is with the School of Automation, Chongqing University, Chongqing 400044, China, and also with the Engineering Product Development Pillar, Singapore University of Technology and Design, Singapore 487372 (e-mail: yan.qin@cqu.edu.cn).

Xiaoli Li is with the Institute for Infocomm Research and the Centre for Frontier AI Research, Agency for Science, Technology and Research (A*STAR), Singapore 138632 (e-mail: xlli@i2r.a-star.edu.sg).

Chau Yuen is with the School of Electrical and Electronics Engineering, Nanyang Technological University, Singapore 639798 (e-mail: chau.yuen@ntu.edu.sg).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TASE.2023.3274648>.

Digital Object Identifier 10.1109/TASE.2023.3274648

1545-5955 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

For instance, Chen et al. [9] developed an attention-enhanced long short-term memory (LSTM) model trained on sensor data to estimate the remaining useful life (RUL) of turbofan engines. Zhang et al. [10] utilized discharging voltage and current data generated from a convolutional recurrent generative adversarial network to build a state-of-health (SoH) estimation model for Li-ion batteries. Zhu et al. [11] proposed a multiscale convolutional neural network (CNN) for the RUL estimation of rolling bearings using vibration signals as model inputs.

However, an underlying assumption in existing studies is that the decentralized data from multiple devices can be centrally pooled for model training. In real industries, aggregating data is often unviable given stringent data privacy laws, such as the General Data Protection Regulation [12], which limits the use and dissemination of sensitive user data. For example, battery health data from electric vehicles (EV) is beneficial to an EV manufacturer for the optimal selection of long-lasting battery types. However, collating battery health data generated from EV drivers is challenging as private user information such as driving patterns may be gleaned from just battery consumption patterns [13]. As a result, the wealth of data collectively held across edge devices cannot be harnessed in practice to train robust health prognosis models, covering a diverse range of practical operations and degradation patterns. Since deep learning models' performance typically increases with data quantity and quality [14], a lack of access to rich edge data may cap productivity gains from intelligent condition monitoring in Industry 4.0 [2], [15].

To meet the demand for privacy-preserving machine learning, several techniques have emerged, and they can be broadly classified into data masking-based or architectural approaches [16]. Data masking is typically utilized when raw data is transmitted to third parties or a central server for model training [17]. A few notable data masking techniques are differential privacy [18], [19], which apply well-designed noise perturbations to obfuscate sensitive individual data attributes, and homomorphic encryption [20], [21], which enable computations to be performed directly on encrypted data without having to decrypt it to the original form first.

In contrast, architectural approaches to privacy-preservation focus on redesigning centralized machine learning paradigms [16]. Among architectural methods, federated learning (FL) [22] has garnered considerable research attention [23], [24], [25], [26] as it facilitates both privacy preservation and decentralized data utilization simultaneously. This makes FL highly relevant for the problem of learning from edge data under privacy constraints, and is thus explored further in this paper. In FL, local data owners, also termed clients, collaboratively train a federated model without disclosing their data. Clients train their own models at the edge using their available data, and only model parameters (i.e., weights and biases) of the client models are communicated to the central server to build the federated model. The data remains with each client, thereby ensuring data privacy [22]. Presently, the most benchmarked FL algorithm is Federated Averaging (FedAvg) [22], where local model parameters are

aggregated by computing a coordinate-wise weighted average of the client models' parameters.

In existing works employing FL for machine condition monitoring, Zhang and Li [27] utilized FedAvg and transfer learning for fault classification of rolling bearings operating under variable conditions. Liu et al. [28] similarly used FedAvg to construct a federated LSTM-based anomaly detector for engine operations. Going beyond fault classification, Rosero et al. [29] applied FL for regression problems, where they compared the performance of federated models based on FedAvg and its variant, FedProx [24] for RUL estimation of turbofan engines. In the aforementioned studies, though the underlying model architectures for the various condition monitoring tasks differed, FedAvg—with its coordinate-wise model parameter averaging scheme—was commonly used for developing the federated model.

However, a shortcoming of FedAvg is that it implicitly assumes coordinate-wise neurons of different client models are learning similar features and hence, they can be averaged. In reality, even with homogeneous datasets across clients, there is no guarantee that the first neuron of a client model is learning the same feature extractors as the first neuron of another client model, and so on [25]. Amidst data heterogeneity, the problem of averaging dissimilar, coordinate-wise neurons is exacerbated, with Wang et al. [26] demonstrating that FedAvg converges to sub-optimal solutions when there are mismatched objective functions of different clients undergoing heterogeneous learning steps.

In a privacy-concerned era, intelligent health prognostics models can benefit significantly from FL because of the exposure to a wider range of operational and machine failure data than any single client's dataset. However, employing FL techniques on industrial data is challenging due to inherent attributes of industrial data. Industrial data, typically sensor data, is noisy and temporal in nature. Consequently, neural networks such as LSTM, which effectively capture important long-term dependencies in long sequence time series [30], tend to have higher prediction accuracies in health prognostics than shallow architectures such as support vector machines [31], [32], [33], and even other standard deep networks such as CNN [31], [34] and gated recurrent units (GRU) [35], [36], [37]. Thus, suitable FL techniques beyond naive parameter averaging are needed to ensure that the weights and biases of hidden states are aggregated meaningfully for the integrity of the overall federated model. Moreover, FL techniques also need to account for heterogeneous data across clients, even for devices of the same type. Data heterogeneity can arise among clients as machines experience dynamic and dissimilar degradation processes in practical conditions, and individual clients may have unequal training data (unbalanced data) from differing machine usage or data collection patterns.

Overall, the crucial research gaps that need to be tackled for reliable industrial health prognostics in a privacy-constrained era are summarized below:

- To realize the theorized benefits of intelligent health prognostics, in-depth FL studies are urgently needed for complex, time series regression-based condition monitoring and health prognosis problems. Otherwise,

rich edge data remain untapped, and productivity gains from intelligent condition monitoring are capped in Industry 4.0 [2], [15].

- Data heterogeneity poses a critical statistical challenge for reliable FL as parameter averaging methods that overlook local heterogeneity dampens the federated model's feature extraction capability and consequently, its predictive performance. Thus, there is a fundamental need for investigating new parameter aggregation algorithms that are suited to both the inherent heterogeneity of degradation data and the recurrent neural networks used for industrial health prognosis.

In view of the challenges posed by client heterogeneity, we postulate that a more suitable parameter aggregation method for industrial health prognostics is one that explicitly considers each client model neuron's feature extraction (i.e., features learnt from input data) and selectively aggregates the neuron parameters based on the similarity of extracted features. Thus, drawing inspiration from the matched averaging (FedMA) algorithm [25] originally introduced for computer vision and language modeling, we develop a pioneering federated model for time series regression-based industrial health prognostics, which efficiently matches similar feature extractions from heterogeneous degradation data for a more discriminating parameter averaging. Specifically, the similarity between client model neurons is determined by their posterior probability distributions, and optimal matching of similar neurons is achieved by solving a neuron assignment cost matrix, formulated as a linear sum assignment problem. Since the FedMA algorithm only averages similar neurons, as opposed to FedAvg's coordinate-wise averaging of potentially dissimilar neurons, the distinct feature extractors of client models are carried over to the federated model with less dilution, thereby boosting its predictive capabilities.

As the seminal FedMA algorithm is designed for complex models with domain-specific components, such as two-dimensional convolution layers or word embedding layers, its applicability to industrial health prognostic models is not straightforwardly clear. Thus, to better suit the lightweight prognostic models used in edge devices, we introduce a customized FedMA algorithm that directly targets the core components of the prognostic model, e.g., temporal hidden states. Furthermore, as we implement the matched averaging of neurons modularly, one model layer at a time, the proposed method copes well with deep architectures, such as LSTM networks that are popularly used for industrial health prognosis. On the whole, our proposed FedMA-based health prognostics model is pivotal for demonstrating how the matched averaging algorithm, previously unconsidered for industrial health prognostics, can be formulated within an end-to-end health prognostics framework, consisting of both offline federated model training and online prognostics on unseen test data.

Overall, the main contributions of this paper are:

- i) We present a systematic empirical investigation of FL algorithms for time series regression-based industrial health prognosis that handles both cyclic and non-cyclic degradation data in a unified framework.

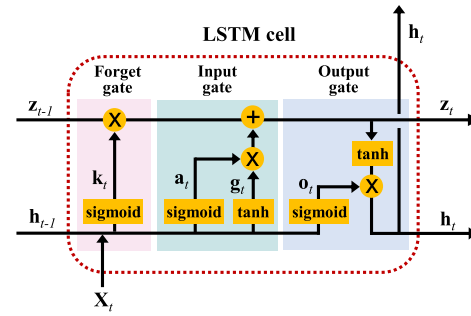


Fig. 1. Inner structure of a LSTM cell and the information flow within it.

- ii) We fundamentally rethink the default coordinate-wise parameter averaging for FL-based health prognostics, and propose a feature similarity-matched averaging scheme for a more discriminating parameter aggregation under practical heterogeneous conditions.
- iii) We validate the effectiveness of our federated prognostic model on two different industrial datasets, achieving robust accuracy improvements as high as 44.5% and 39.3% for SoH and RUL estimation, respectively. Thus, our work is pivotal for spurring research into effective parameter aggregation methods for FL-based industrial health prognostic models.

The remainder of the paper is organized as follows. Section II describes some preliminary information for a better understanding of industrial health prognosis and federated learning. Section III formulates the proposed federated prognostic model with the feature similarity-matched parameter averaging scheme. Section IV discusses the dataset, experimental formulation, and results. Section V concludes and offers future research directions.

II. PRELIMINARY

To aid the understanding of the advanced FL-based prognostic models proposed in the paper, this section discusses some key background knowledge on the quintessential health prognostic tasks encountered for cyclic and non-cyclic degradation patterns, the basic LSTM architecture utilized for health modeling, and the standard federated model development process.

A. Health Prognosis for Cyclic and Non-Cyclic Degradation

Degradation patterns can be broadly classified into cyclic and non-cyclic. Cyclic degradation refers to non-monotonic and repetitive degradation behavior from cyclical stresses. For instance, Li-ion batteries experience cyclic capacity degradation due to electrochemical losses from repeated charging and discharging operations [38]. Thus, a quintessential health prognosis task for batteries is SoH estimation, which predicts a battery's practically achievable capacity for future operating cycles till its end-of-life.

In contrast, non-cyclic degradation patterns are monotonous and non-repetitive. A well-studied example is the piecewise degradation modeling of turbofan engines [9], where degradation is negligible during initial operations, but starts to occur

continuously after some time in operation. For these industrial assets, RUL estimation is a health prognosis task of significant interest, where RUL is defined as the remaining length of operational time until an asset's complete failure.

B. LSTM Neural Network for SoH and RUL Estimation

As an architecture well-suited for long sequence prediction, LSTM is commonly used in health prognostic models of critical assets, such as engines [31], [32], bearings [39], [40], [41], and batteries [42], [43], [44]. The effectiveness of LSTM is due to its ability to keep track of important sequential dependencies in input data, even across long time periods [30]. Compared to the structurally simpler GRU network [45], LSTM also performs better on complex sequences and learning tasks due to its enhanced information retention abilities [46], [47], [48]. Thus, LSTM is used as the backbone for the proposed SoH and RUL estimation models in this paper.

A basic LSTM cell is an information holding unit with three gates (forget, input, and output) that allow important information to be propagated across time and irrelevant information to be forgotten. As shown in Fig. 1, at each time t , the current input sequence $\mathbf{X}_t$ and the previous hidden state $\mathbf{h}_{t-1}$ are fed into gates with sigmoid or tanh functions to yield intermediate outputs, $\mathbf{k}_t$, $\mathbf{a}_t$, $\mathbf{g}_t$, and $\mathbf{o}_t$. As the elements of these outputs range from 0 to 1 (for sigmoid activation) and -1 to 1 (for tanh activation), they are essentially scaled values indicating the relative importance of the information in $\mathbf{X}_t$ and $\mathbf{h}_{t-1}$. These values are then used to scale the relative contribution of the long-term information that is held in the previous cell state $\mathbf{z}_{t-1}$ to determine the new cell state $\mathbf{z}_t$ and hidden state $\mathbf{h}_t$, as follows:

$$\begin{aligned}\mathbf{z}_t &= \mathbf{k}_t \times \mathbf{z}_{t-1} + \mathbf{a}_t \times \mathbf{g}_t \\ \mathbf{h}_t &= \mathbf{o}_t \times \tanh(\mathbf{z}_t)\end{aligned}\quad (1)$$

where $\times$ refers to element-wise multiplication.

The final predicted output $\hat{y}_t$ (e.g., RUL values) is obtained from $\hat{y}_t = \sigma(\mathbf{h}_t)$, where $\sigma(\cdot)$ refers to fully connected regression layers with appropriate activation functions.

C. Federated Learning

The goal of FL is to build a shared federated model that is attuned to local client-specific features and yet, also general enough to handle the diverse practical conditions not seen by a local training dataset. Since our focus is *horizontal FL* [12], where client data have the same feature and target variables but differ in the sample space, the same model architecture can be adopted for all client models and correspondingly, the federated model. The standard FL process is summarized as a high-level schematic in Fig. 2, and discussed below, using the seminal FedAvg algorithm as an example.

- 1) Local clients independently train their individual local models for E epochs with their private data.
- 2) The trained local model parameters are communicated to a central server, and aggregated to form the parameters of a new federated model or update an existing one.

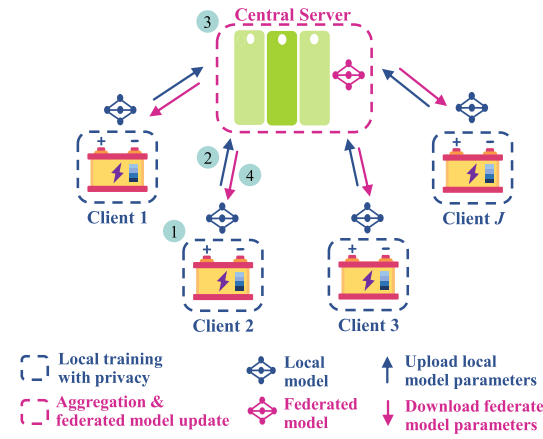


Fig. 2. Key steps of federated learning: ① local model training, ② local model parameter upload, ③ parameter aggregation to form federated model, and ④ adoption of federated model by clients.

- 3) For FedAvg, the local model parameters are averaged by computing a coordinate-wise mean, weighted by the proportion of the client dataset size:

$$\mathbf{W}^r = \sum_{j=1}^J p_j \mathbf{W}_j^r \quad (2)$$

where $\mathbf{W}_j^r$ is client j 's model parameters at communication round r , p_j is the fraction of total data samples belonging to client j , and $\mathbf{W}^r$ is the federated model parameters.

- 4) The federated model parameters $\mathbf{W}^r$ are broadcasted back to the local clients, and their models are initialized with the federated model parameters.

III. FL-BASED INDUSTRIAL HEALTH PROGNOSTICS USING MATCHED FEATURE EXTRACTION

The proposed FedMA-based health prognostics framework consists of two stages—offline federated model training with idle local clients and online inferencing using the trained model. Individual clients kickoff the offline modeling process by training their respective initial client models at the edge, using only their private data. To then build the federated model, a layer-wise matched averaging algorithm is applied to the initial client models, where the hidden neurons of different clients are matched and aligned based on similar feature extraction traits first before their weights and biases are averaged. These averaged parameters become the federated model's parameters, which are subsequently distributed back to the clients for further rounds of collaborative federated model training. During online inferencing, clients employ the trained federated model for health prognosis tasks on incoming private test data. The steps of the proposed method are summarized in Fig. 3.

A. Local Client Model Formulation

As the local model architecture serves as the basis for the federated model in *horizontal FL*, we begin with the local model formulation for some quintessential health prognosis tasks, e.g., SoH estimation or RUL estimation. Since the model

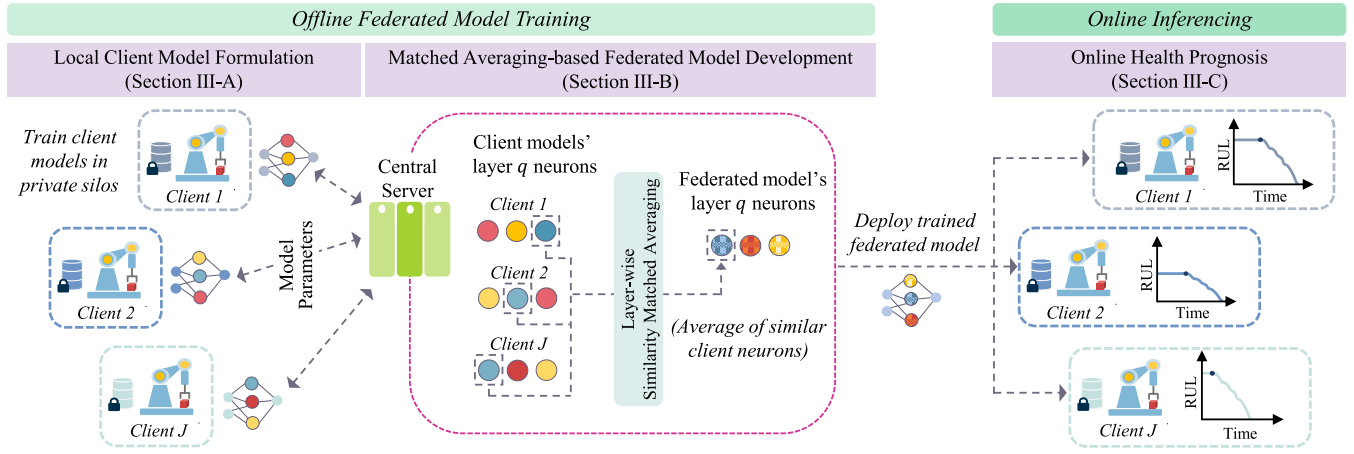


Fig. 3. Overview of proposed FedMA-based health prognosis framework.

TABLE I
PARAMETERS OF A Q-LAYER LSTM-BASED MODEL ARCHITECTURE

Layer index	Parameter, $W_{j,q}$	Shape ^a
1	lstm.weight.input-hidden	$(U * L, D_{in})$
	lstm.weight.hidden-hidden	$(U * L, L)$
	lstm.bias.input-hidden	$(U * L)$
	lstm.bias.hidden-hidden	$(U * L)$
q	$\vdots$	$\vdots$
Q	regressor.weight	(D_{out}, L)
	regressor.bias	(D_{out})

^a U , D_{in} , L , and D_{out} are, respectively, the number of intermediate LSTM cell weights, input features, hidden neurons, and output samples.

architecture for each type of prognostic task is the same across all clients (only parameter values differ), a general Q -layer model is discussed from a single client j 's perspective. Here, Q is the cardinality of the set of layer indexes for a general LSTM-based architecture depicted in Table I.

1) *SoH Estimation With Cyclic Degradation*: Assuming M process variables (e.g., voltage, temperature of discharging battery) are measured at discrete time indexes $t \in \{1, \dots, T_s\}$ for each operating cycle s in the cyclic data of client j , a feature data matrix $\mathbf{X}_s \in \mathbb{R}^{M \times T_s}$ can be formed, where T_s is the varying cycle length of different cycles. To develop an LSTM-based SoH estimation model, the feature data matrix has to be transformed into inputs that are suited for the LSTM architecture. First, the feature data are standardized to have zero mean and unit variance with respect to each process variable so that extreme values do not skew model training and convergence. Next, as the LSTM architecture requires fixed-length input sequences, the standardized feature data are segmented into smaller sequences determined by the shortest cycle to become $\bar{\mathbf{X}}_s \in \mathbb{R}^{M \times T}$, where scalar $T = \min\{T_s\}_{s=1}^S$. The health indicator label (e.g., battery capacity) at the end of each cycle is denoted as Y_s .

The pairwise sequences $(\bar{\mathbf{X}}_s, Y_s)$ are then consecutively fed into an LSTM network to train client j 's SoH estimation model, $\hat{Y}_s = \varphi(\bar{\mathbf{X}}_s, \mathbf{W}_j)$, where φ is a non-linear function and $\mathbf{W}_j = \{\mathbf{W}_{j,q}\}_{q=1}^Q$, the set of model parameters for the Q layers. The optimal $\mathbf{W}_j$ is obtained by minimizing the

estimation error:

$$\min_{\mathbf{W}_j} \sum_{s=1}^S \left\| Y_s - \varphi(\bar{\mathbf{X}}_s, \mathbf{W}_j) \right\|_2^2 \quad (3)$$

2) *RUL Estimation With Non-Cyclic Degradation*: For non-cyclic degradation, the progression of degradation occurs continuously across time, and there are no clear repetitive degradation patterns across operational cycles. Thus, the feature data matrix is represented without any reference to cycles as $\mathbf{X} \in \mathbb{R}^{M \times T}$, where T is length of the time series measurements of the M process variables. For run-to-failure data, T is equivalent to the lifespan of the device. The feature data pre-processing steps of standardization (zero mean and unit variance) and data segmentation into smaller, fixed-length sequences are similarly performed.

For the data segmentation, a sliding window approach is utilized since non-cyclic degradation data need not be segmented by cycles [49]. A sliding window of length λ is shifted through the entire time series with time step size $\Delta\tau = 1$ to generate smaller sequences $\tilde{\mathbf{X}}[\tau : \tau + \lambda]$, starting from time index τ and of length λ . The corresponding health indicator label (i.e., RUL) at the end of the feature data sequence is given as $Y[\tau + \lambda]$. The pairwise segmented sequences, still denoted as $(\tilde{\mathbf{X}}, Y)$ for brevity, are consecutively fed into the LSTM network to train client j 's Q -layer RUL estimation model, $\hat{Y} = \varphi(\tilde{\mathbf{X}}, \mathbf{W}_j)$, where $\mathbf{W}_j$ is once again the set of model parameters for the Q layers. The optimal $\mathbf{W}_j$ is obtained by minimizing the estimation error, similar to Eq. 3.

B. Matched Averaging-Based Federated Model Development

To build a potentially higher performing health prognostic model than any single local model trained on limited private data, a Q -layer federated model that learns from the optimal parameters $\{\mathbf{W}_j\}_{j=1}^J$ of J clients is developed in this section. To aggregate the local model parameters meaningfully in the presence of client heterogeneity, neurons of different client models are considered layer-by-layer and matched based on their feature extraction similarity first before their parameters

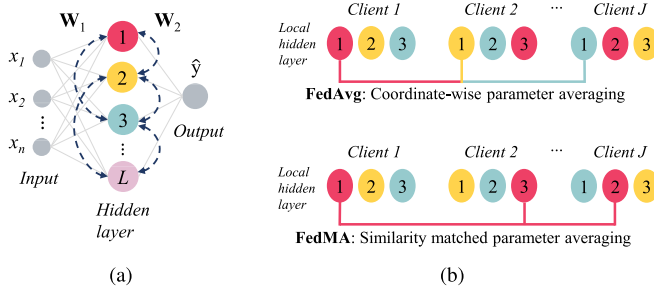


Fig. 4. (a) A basic fully connected neural network has $L!$ equivalent parametrizations due to the permutation invariance of parameter ordering. (b) Visual comparison of FedAvg vs. FedMA parameter averaging scheme, where similar neurons of different clients are represented by the same color.

are averaged to form the federated model's parameters. The mathematical formulation for the similarity-based neuron matching and the layer-wise matched averaging process for the federated model development are discussed below.

1) *Similarity-Based Neuron Matching*: The need for matching neurons based on similar feature extraction traits arises due to permutation invariance of neural network parameters. For a given network, there exists several equivalent variants that differ in the ordering of model parameters but produce the same output value [25]. For example, in the basic fully connected client model illustrated in Fig. 4(a), $\hat{y} = \sum_{i=1}^L \mathbf{W}_{j,2,i} \cdot \sigma(\langle \mathbf{x}, \mathbf{W}_{j,1,i} \rangle)$, where $\cdot i$ and $i \cdot$ are the i -th column and i -th row of weights $\mathbf{W}_{j,1}$ and $\mathbf{W}_{j,2}$ respectively, σ is a non-linear activation function, and L is the number of hidden layer neurons. As the summation operation is permutation invariant, $L!$ equivalent parametrizations are possible for any $\{\mathbf{W}_{j,1}, \mathbf{W}_{j,2}\}$ by varying the parameter order. Given the numerous permutations available, the coordinate-wise elements of different local clients' weight vectors may not necessarily be representing similar feature extraction traits, even for homogeneous local datasets. Thus, the coordinate-wise parameter averaging of the FedAvg algorithm may lead to lower federated model performance due to dissimilar feature extractors (neurons) being averaged and the resultant dampening of feature extraction capabilities. Instead of coordinate-wise parameter averaging, FedMA matches the neurons of local client models based on their similarity first before averaging them to form the federated model neurons. Fig. 4(b) visually compares the key difference between the FedAvg and FedMA parameter averaging scheme.

The similarity between the local client model neurons as well as the to-be-inferred federated model neurons is formulated by the similarity function $c(w_{jl}, \theta_i)$, where w_{jl} and θ_i are the respective weight vectors of client model j 's l -th neuron and the federated model's i -th neuron. To allow for the nonparametric discovery of the federated model neurons, the federated model is framed as a Bayesian probabilistic model that follows the Beta-Bernoulli process (BBP) [25]. To compute the maximum a posteriori estimate (MAP) of θ_i , the similarity function $c(w_{jl}, \theta_i)$ is further defined to be the posterior probability of neuron l from client model j drawn from a Gaussian distribution with mean θ_i . Similarity matching of the client model neurons then requires solving the objective

function in Eq. 4 to obtain the required permutations, i.e., neuron matching assignments, $\{\pi_{li}^j\}$ that match the indexes of local client model neurons based on their similarity:

$$\begin{aligned} \min_{\{\pi_{li}^j\}} \quad & \sum_{i=1}^L \sum_{j,l} \min_{\theta_i} \pi_{li}^j c(w_{jl}, \theta_i) \\ \text{s.t.} \quad & \sum_i \pi_{li}^j = 1 \quad \forall j, 1 \leq l \leq L_j; \\ & \sum_l \pi_{li}^j = 1 \quad \forall j, 1 \leq i \leq L. \end{aligned} \quad (4)$$

where, the constraints on π_{li}^j are needed to form permutation matrices mathematically.

In practice, it is possible that not all local model neurons can be well matched across clients. Instead of forcing poor matches, the unmatched neuron is added as a new neuron to the federated model. At the same time, excessive federated model growth is limited by a penalty function $f(\cdot)$ that increases with model size. Thus, to solve for the required permutation $\{\pi_{li}^{j'}\}_{l,i}$ for a client j' , the objective function in Eq. 4 is expanded to the general form:

$$\begin{aligned} \min_{\{\pi_{li}^{j'}\}_{l,i}} \quad & \mathbf{P} = \sum_{i=1}^{L+L_{j'}} \sum_{l=1}^{L_{j'}} \pi_{li}^{j'} C_{li}^{j'} \\ \text{s.t.} \quad & \sum_i \pi_{li}^{j'} = 1 \quad \forall l; \\ & \sum_l \pi_{li}^j \in \{0, 1\} \quad \forall i, \end{aligned} \quad (5)$$

$$\text{where } C_{li}^{j'} = \begin{cases} c(w_{jl}, \theta_i), & i \leq L \\ \epsilon + f(i), & L < i \leq L + L_{j'} \end{cases}$$

The matching cost threshold ϵ and penalty function $f(\cdot)$ are governed by the Indian Buffet Process prior [50], which models the intuitive phenomenon that if a local neuron is popularly observed in most client models, then there is higher probability for it to be matched and adopted as a global neuron. Further details and proof on the matching cost and penalty formulation are found in [25] and [51].

The optimization problem in Eq. 5 is a linear sum assignment problem [51], thus, the Hungarian matching algorithm [52] is applied iteratively over all clients to solve for $\{\pi_{li}^{j'}\}_{l,i}$, keeping other permutations $\{\pi_{li}^j\}_{l,i,j \neq j'}$ fixed. For each iteration, the corresponding federated model $\{\theta_i = \arg \min_{\theta_i} \sum_{j \neq j', l} \pi_{li}^j c(w_{jl}, \theta_i)\}_{i=1}^L$ is determined, based on current estimates of $\{\pi_{li}^j\}_{l,i,j \neq j'}$. When faced with a client model j' with $L_{j'}$ neurons, the federated model is updated by applying the Hungarian algorithm to match the current federated model to candidate local neurons $\{w_{j'l}\}_{l=1}^{L_{j'}}$. The new federated model size L' is controlled by the earlier discussed penalty cost $f(\cdot)$ and limited to $L \leq L' \leq L + L_{j'}$. This nonparametric federated model framework and the steps to solve Eq. 5 are termed BBP-MAP henceforth.

2) *Federated Health Prognostic Model Development With FedMA*: Building on the neuron matching formulation,

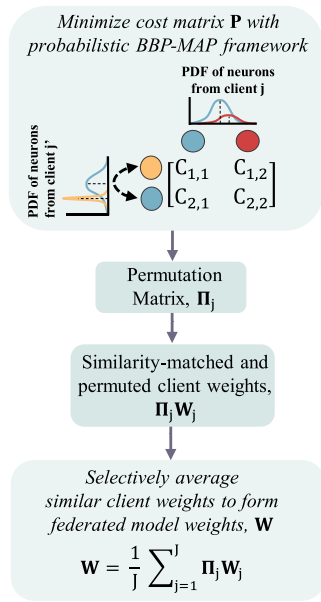


Fig. 5. Similarity-matched averaging scheme for federated model development.

we discuss a layer-wise framework for matching and averaging the client model parameters to construct the Q -layer, LSTM-based federated model and determine its parameters. As performing matched averaging recursively through deeper model layers deteriorates the federated model performance [25], a modular, layer-wise approach is adopted, where only the q -th layer parameters $\{\mathbf{W}_{j,q}\}_{j=1}^J$ of the J client models are considered to perform matched averaging and compute the corresponding q -th layer parameters $\mathbf{W}_q$ of the federated model. This one-layer matched averaging scheme is repeated for all layers, beginning with all the client models' first layers (i.e., $q = 1$ in Table I) and working down to their last output layers (i.e., $q = Q$). The detailed layer-wise federated model development is outlined below and depicted as a schematic in Fig. 5.

Following the local model training described in Section III-A, the clients' q -th layer parameters $\{\mathbf{W}_{j,q}\}_{j=1}^J$ are gathered by the central server to begin the neuron matching process across clients. The q -th layer parameters $\mathbf{W}_{j,q}$ of each client j are transformed to form inputs to Eq. 5 as the concatenated vector of parameters (weights and biases) associated with each of L_j hidden neurons $\{w_{jl} \in \mathbb{R}^{U_{D_{in}}+U}\}_{l=1}^{L_j}$, where U is the number of intermediate LSTM cell weights (i.e., 4) [30] and D_{in} is the input size. These weight vectors are iteratively fed into the probabilistic BBP-MAP cost formulation discussed in Section III-B(1) to solve Eq. 5 for the required index assignments $\{\pi_{li}^j\}$, rewritten as Π_j^T , that match client j 's neurons to similar neurons of all other clients excluding j .

Within each LSTM layer, there are both the input-to-hidden and hidden-to-hidden components as shown in Table I. For input-to-hidden weights $\mathbf{W}_{j,q}$, the matched index assignments Π_j^T are multiplied to the right (acts on columns) of the original client weights $\mathbf{W}_{j,q}$ to yield the correctly permuted

Algorithm 1 FedMA-Based Health Prognostic Model Development

Input: Local parameters $\{\mathbf{W}_{j,1}, \dots, \mathbf{W}_{j,Q}\}_{j=1}^J$ of J client models with a Q -layer architecture

Output: Federated model parameters $\{\mathbf{W}_1, \dots, \mathbf{W}_Q\}$

Initialize layer index $q = 1$

```

1: while ( $q \leq Q$ ) do
2:   if ( $q < Q$ ) then
3:      $\{\Pi_j\}_{j=1}^J = \text{BBP-MAP}(\{\mathbf{W}_{j,q}\}_{j=1}^J)$ ;
4:      $\mathbf{W}_q = \frac{1}{J} \sum_j \mathbf{W}_{j,q} \Pi_j^T$ ;
5:   else
6:      $\mathbf{W}_q = \frac{1}{J} \sum_j \mathbf{W}_{j,q}$ ;
7:   end if
8:   for  $j \in \{1, \dots, J\}$  do
9:      $\mathbf{W}_{j,q} \leftarrow \mathbf{W}_q$ ;
10:     $\mathbf{W}_{j,q+1} \leftarrow \Pi_j \mathbf{W}_{j,q+1}$ ;
11:    Retrain  $\{\mathbf{W}_{j,q+1}, \dots, \mathbf{W}_{j,Q}\}$  with  $\mathbf{W}_q$  frozen;
12:   end for
13:    $q = q + 1$ 
14: end while

```

client weights $\mathbf{W}_{j,q} \Pi_j^T$, where similar neurons now hold the same index position across different clients' weight matrices. The weights $\mathbf{W}_q$ for the federated model's q -th layer is then computed by averaging the similarity-matched q -th layer parameters of the J client models:

$$\mathbf{W}_q = \frac{1}{J} \sum_{j=1}^J \mathbf{W}_{j,q} \Pi_j^T \quad (6)$$

For the hidden-to-hidden parameters $\mathbf{H}_{j,q}$, both the rows and columns of $\mathbf{H}_{j,q}$ are permutable, thus, the solved permutation matrix Π_j is applied to the left and right of $\mathbf{H}_{j,q}$. The hidden-to-hidden parameters $\mathbf{H}_q$ for the q -th layer of the federated model is then computed by averaging the similarity-matched hidden layer weights of the J client models:

$$\mathbf{H}_q = \frac{1}{J} \sum_{j=1}^J \Pi_j \mathbf{H}_{j,q} \Pi_j^T \quad (7)$$

In the current layer-wise approach, once the q -th layer federated parameters $\mathbf{W}_q$ (includes both input-to-hidden and hidden-to-hidden parameters for simplicity) are computed, it is distributed back to the local client models first, to be initialized as their respective q -th layer parameters. Keeping their initialized q -th layers frozen, clients retrain all subsequent layers of their models on their private datasets to generate updated parameters for the remaining layers based on the q -th layer initialization. This process of conducting similarity-based matched averaging on a selected q -th layer of client models to determine the federated model layer, followed by local client model retraining of subsequent layers is continued until the last output layer. For the last layer, there is no ambiguity in the ordering of the output nodes (the permutation matrix is, in fact, an identity matrix). Thus, a simple mean of the last layer local model parameters yields the federated model's last layer parameters. The complete FedMA round to determine

TABLE II

EXPERIMENTAL SETTINGS TO ASSESS ROBUSTNESS OF FL TECHNIQUES

Experiment 1: SoH estimation of batteries	Experiment 2: RUL estimation of turbofan engines
- Cyclic degradation data - Cross-device FL within the same organization - Heterogeneous local devices with dissimilar degradation processes	- Non-cyclic degradation data - Cross-organization FL where each organization owns multiple devices - Heterogeneous organizations with dissimilar degradation processes among groups of devices and unbalanced training datasets

all parameters $\mathbf{W} = \{\mathbf{W}_q\}_{q=1}^Q$ of the Q -layer federated model is summarized in Algorithm 1. Once the first iteration of the federated model is constructed, the model performance can be further improved by repeating the FedMA procedure for more communication rounds between the server and client models until a desired accuracy or model convergence is achieved.

C. Online Health Prognosis

Once the federated model is sufficiently trained offline, it is deployed back to local clients, which now have a health prognosis model that has been exposed to a wider range of operational and machine failure patterns despite local data limitations and privacy constraints. With the federated model, clients can individually conduct online inferencing tasks on incoming private test data $\mathbf{X}_{test}$. For SoH estimation with cyclic test data and RUL estimation with non-cyclic test data, the pre-processing transformations outlined in the respective Sections, III-A(1) and III-A(2) are applied to $\mathbf{X}_{test}$ prior to inputting it into the LSTM-based federated model, $\hat{Y}_{test} = \phi(\mathbf{X}_{test}, \mathbf{W})$. Here, ϕ is a general, non-linear representation of either an SoH estimation or RUL estimation model, $\mathbf{W}$ is the trained federated model parameters from Section III-B(2), and $\hat{Y}_{test}$ is the predicted SoH or RUL value.

IV. EXPERIMENTS AND RESULTS

This section evaluates the SoH and RUL estimation performance of FedMA-based models against the standard FedAvg-based models and importantly, the baseline local models without FL. Using two distinctly different datasets (Li-ion batteries and turbofan engines), the benchmarkable experiments have been thoughtfully designed to assess the generalizability and robustness of the studied FL methodologies in a variety of settings such as i) cyclic vs. non-cyclic degradation data, ii) cross-device FL within the same organization vs. cross-organization FL, and iii) device-level heterogeneity vs. organization-level heterogeneity. Table II summarizes the settings examined for each experiment. In addition to FL performance analysis, each experiment also discusses practical insights, such as the potential for training data efficiency in Experiment 1 and the interpretability of similar feature extraction traits in Experiment 2.

A. Experiment 1: SoH Estimation of Li-Ion Batteries

In this problem formulation, each battery is a client requiring an accurate, on-device SoH estimation model. Under

TABLE III

DESCRIPTION OF NASA BATTERY DATASET

Battery	Cut-off voltage	Battery life cycles before EoL	Training cycles (70% split)	Test cycles (30% split)
B0006	2.5 V	121	85	36
B0007	2.2 V	168	118	50
B0018	2.5 V	122	85	37

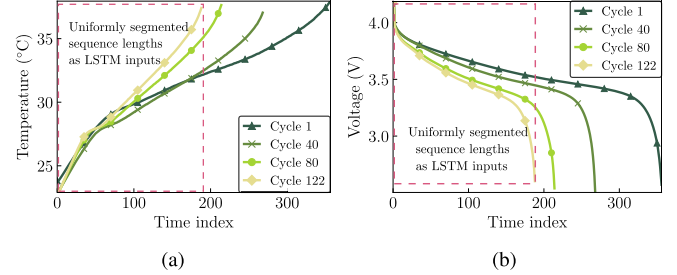


Fig. 6. (a) Temperature signals, and (b) voltage signals of the battery B0018 from the NASA dataset.

privacy constraints, clients have to develop their local models independently using only their respective datasets for model training. However, with FL, a richer federated model can be built through collaborative learning from multiple client models and utilized back by the clients. As the batteries are all of the same type, this experiment is akin to a real-world, cross-device FL setup within a single organization. The accuracy of the resultant federated model is evaluated.

1) *Data Description*: The NASA battery dataset [53] consists of 2 Ah-rated Li-ion cells that have undergone repeated charging and discharging cycles in accelerated ageing experiments. Conventionally, it is the discharging cycle that is relevant to SoH estimation as the discharging voltage, current, and temperature measurements form the input features for the capacity estimation model. The batteries were discharged from a voltage of 4.2V to a cut-off voltage of either 2.2V or 2.5V at a constant 2A current and an ambient temperature of 24 °C (though the actual temperature of the battery varies as it heats up during the charge-discharge operations). Taking battery B0018 as an example, Fig. 6 illustrates the progression of the measured voltage and temperature time series within each discharge cycle for selected discharge cycles. Generally, the length of a discharge cycle, given by the length of the time index, decreases from the first cycle to the last.

As the discharging cycles progress, capacity degradation also occurs until the end-of-life (EoL) capacity, defined as 70% of original rated capacity. Only battery data up till EoL are used for SoH estimation modeling, with the first 70% of cycles allocated for model training and the remaining 30% for testing. The salient attributes of the dataset are summarized in Table III. To examine the battery degradation patterns further, the capacity loss over the discharge cycles is plotted for each battery in Fig. 7. It is evident that the capacity degradation follows a cyclic pattern—amidst the overall decreasing trend, there exists periodic localized peaks due to the capacity regeneration phenomenon [54]. Secondly, the three batteries exhibiting notably different capacity degradation pathways despite having similar discharging

TABLE IV

HYPERPARAMETER SELECTION FOR LOCAL MODEL ARCHITECTURE

Hyperparameters	Search space	Selected value (Experiment 1)	Selected value (Experiment 2)
Sequence length	{171}, {30, 40, 50}	171 ^a	50
LSTM layers	{1, 2}	1	1
Neurons per layer	{50, 128, 256, 300}	128	256
Learning rate	{0.0005, 0.001, 0.005}	0.001	0.001
Optimizer	RMSProp, Adam	Adam	Adam

^a Sequence length is determined by the length of the shortest discharge cycle across all batteries.

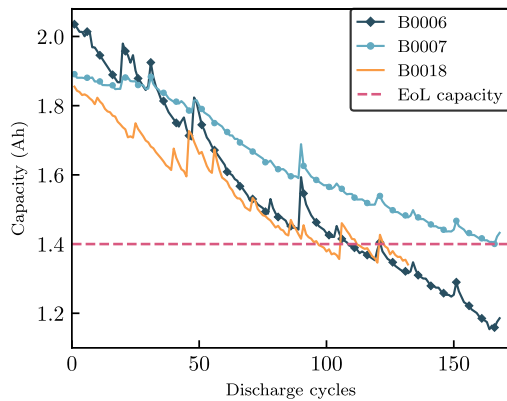


Fig. 7. Capacity loss over cycles for batteries, ‘B0006’, ‘B0007’, and ‘B0018’.

conditions reiterates the need for FL techniques that account for local data heterogeneity.

2) Local Client Model Formulation for SoH Estimation:

First, the local model for each client battery is developed. Using only the first 70% of a battery’s own discharge cycles as training data, a LSTM-based SoH estimation framework is built to model the non-linear relationship between the feature variables, discharging voltage and temperature, and the capacity labels. The discharging current is not selected as a feature as the batteries were discharged at constant current and thus, the current measurements do not provide useful information about the capacity degradation process. Two key pre-processing steps are performed on the features to transform them as suitable LSTM inputs. Features are standardized to have zero mean and unit variance so that large values do not dominate and skew model training and convergence. As the LSTM model requires fixed length input sequences, the standardized feature time series are then segmented into sequences of a common length determined by the shortest discharge cycle as depicted in Fig. 6. These feature sequences, together with their corresponding capacity labels, are consecutively fed into the LSTM framework to model the progression of capacity levels through the training discharge cycles and estimate the capacity, or equivalently, SoH for each battery’s test data (i.e., the earlier partitioned last 30% of discharge cycles). For the local model architecture, a lighter model is preferred for a lower communication burden between the server and the clients during FL. Keeping this in mind, the model variations listed in Table IV were assessed. All models were trained for 100 epochs with early stopping. A one-layer LSTM network with 128 neurons,

followed by a fully connected output layer (therefore, $Q = 2$), emerged as the best-performing SoH estimation model for the test data. Each client’s best local model accuracy represents the maximum achievable performance by each client under privacy constraints without any FL and it forms the baseline for comparing the FL performance later in Section III-A(4).

3) Federated Model Development for SoH Estimation:

Using the same 2-layer architecture as the local models, the federated models are developed next using the FedAvg and FedMA algorithms. The FedAvg-based federated model is constructed following the steps in Section II-C, where the local model parameters are aggregated coordinate-wise using Eq. 2. According to the FedAvg algorithm, the retraining of local models following their initialization with the federated model parameters has to be performed incrementally for E epochs every communication round to reduce the possibility of FL training overshooting and diverging [25]. As the local models previously needed about 100 epochs for convergence in the baseline scenario without FL, $E \in \{2, 5, 10\}$ were considered as candidate epochs for the incremental training. The best federated model performance was achieved with $E = 2$ for local model retraining. In contrast, the FedMA algorithm favors longer retraining and convergence of local models before the local model parameters are aggregated. Sufficiently converged local models ensure that the similarity matching of neurons are performed based on stable, high quality feature extraction traits. The FedMA-based federated model is constructed following Algorithm 1, with local model retraining conducted for 120 epochs within each communication round. Compared to the 100 epochs used for local model training in the baseline scenario without FL, the local model retraining for FedMA is understandably slightly longer since the collaborative learning among multiple clients exposes more degradation patterns to be learnt.

4) SoH Estimation Results and Discussion: The FedMA and FedAvg federated models were both trained up to 20 communication rounds. For each round, the RMSE of the capacity predictions was computed to evaluate the federated model accuracy on local clients’ test data. The best federated model accuracy, the number of communication rounds needed, and the accuracy improvement over baseline client models without any FL are presented in Table V. Generally, the federated models outperformed client models trained on only their own data, which reiterates the crucial role FL plays in allowing local client models to benefit from a wider range of training data despite privacy constraints. Comparing the two FL methods, FedMA achieved at least 1.5 times larger accuracy improvements than FedAvg for all client batteries. While FedAvg needed less communication rounds to reach its best model, it also converged to sub-optimal results than FedMA due to its coordinate-wise parameter aggregation algorithm, which ignores local data heterogeneity.

Against the known premise that more training data leads to better model performance, the scope for **data-efficiency** in FL was also explored since utilizing less clients offers practical advantages such as reduced communication burden and training time. For FedMA models, training on a smaller select subset of clients was viable, as shown in Table IV. Not

TABLE V
PERFORMANCE COMPARISON OF FL MODELS AGAINST LOCALLY TRAINED MODELS AND CENTRALLY TRAINED MODELS WITH POOLED DATA

Test Client Battery	Train Client Batteries	Method	RMSE	Best communication round	IMP ^a
B0006	B0006	Local data only	0.04979	-	-
	B0006, B0007, B0018	Central	0.05307	-	-
	B0006, B0007, B0018	FedAvg	0.04588	4	7.9 %
	B0006, B0007, B0018	FedMA (proposed)	0.04381	15	12.0 %
	B0006, B0018	FedMA (proposed)	0.03871	4	22.3 %
B0007	B0007	Local data only	0.03610	-	-
	B0006, B0007, B0018	Central	0.03296	-	-
	B0006, B0007, B0018	FedAvg	0.02933	4	18.8 %
	B0006, B0007, B0018	FedMA (proposed)	0.02502	16	30.7 %
	B0006, B0007	FedMA (proposed)	0.02193	5	39.2 %
B0018	B0018	Local data only	0.03361	-	-
	B0006, B0007, B0018	Central	0.02972	-	-
	B0006, B0007, B0018	FedAvg	0.02868	4	14.7 %
	B0006, B0007, B0018	FedMA (proposed)	0.02257	16	32.9 %
	B0006, B0018	FedMA (proposed)	0.01865	14	44.5 %

^a IMP is short for improvement, which is defined as ('Local data only' $RMSE$ - FL $RMSE$) / 'Local data only' $RMSE$.

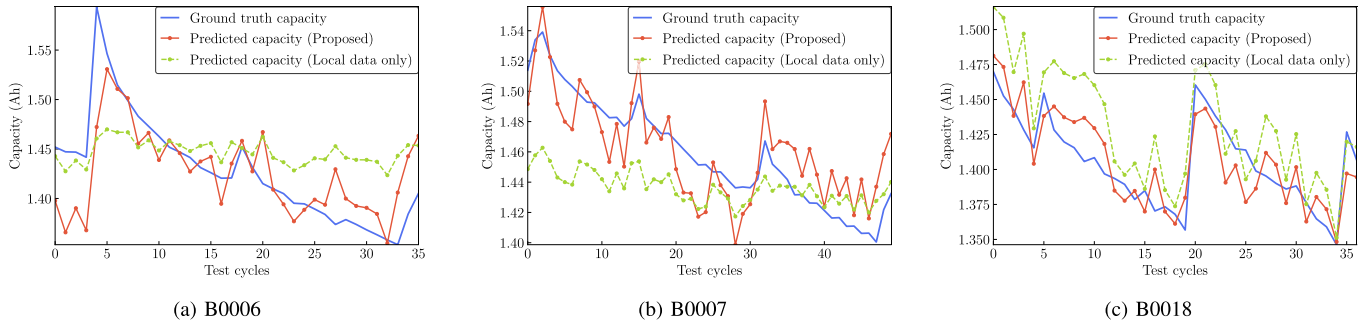


Fig. 8. Predictive performance of local model without any FL collaboration (local data only) vs. proposed approach for test cycles from batteries (a) B0006, (b) B0007, and (c) B0018.

only did the federated model accuracy significantly increase (lowest $RMSE$), the communication rounds required to achieve the best model also decreased. In contrast, FedAvg models generally worsened when trained on smaller client subsets (the additional FedAvg results are not reported for brevity). A possible reason for FedMA's notable outperformance in this data-efficient scenario is that the beneficial presence of certain compatible clients is amplified by the matching of similar client neurons. The resultant aggregated federated model has sharper feature extraction capabilities and hence, accuracy. For an in-depth look into the SoH estimation performance of the FedMA federated model, Fig. 8 plots the predicted capacities against the ground truth labels for each client battery's test data. Compared to the baseline client models trained on only their limited data, FedMA's predictions exhibit remarkable closeness to ground truth capacities and their trend.

As an added evaluation criterion, the FL models are compared against the ideal setting, where there are no privacy constraints and edge data can be directly pooled for central training. This central training scenario should theoretically have the best model performance (and forms the upper bound for FL performance) because it is able to directly utilize all client data instead of relying on model parameter aggregation. As expected, the 'Central' models, reported in Table V, outperformed most client batteries' local models trained only on local data. However, contrary

to initial expectations, the 'Central' models consistently underperformed FL models, revealing a noteworthy advantage of FL. The underperformance of the 'Central' models in this experiment is likely due to the cyclic nature of battery capacity degradation data (as previously seen in Fig. 7). As opposed to non-cyclic, monotonously decreasing data, centrally aggregating cyclic data of different batteries challenges the model's ability to distinguish between regular cyclic patterns of the current battery and the start of the next battery's discharge cycles. In contrast, FL models consider each battery's model and indirectly, the data separately. Thus, they can reap the benefit of learning from multiple client batteries without being disturbed by cyclic noise.

B. Experiment 2: RUL Estimation of Turbofan Engines

The second experiment is designed to investigate the scalability of FL methods in a cross-organization setting, where each organization (client) owns a heterogeneous sample of training devices but needs to conduct RUL estimation for some test devices beyond the distribution of their training samples. Heterogeneity, in the form of unequal amount of training data samples and devices with dissimilar degradation processes, is introduced to clients. The RUL estimation performance of the FedMA and FedAvg models amidst the data heterogeneity is assessed.

TABLE VI
PERFORMANCE COMPARISON OF FL MODELS AGAINST LOCAL MODELS AND CENTRALLY TRAINED MODELS WITH POOLED DATA

Setting	Clients	No. of train engines	No. of test engines ^a	RMSE			Best communication round		IMP ^b
				Local data only/ Central pooled	FedAvg	FedMA (proposed)	FedAvg	FedMA (proposed)	
Heterogeneous partitioning	HT1 (<200 cycles)	82	228	39.26	28.61	27.22	9	4	30.7%
	HT2 (200-350 cycles)	25	228	44.87					1.9%
	HT3 (> 350 cycles)	142	228	27.76					39.3%
Homogeneous partitioning	HO1	83	228	32.94	30.42	30.09	13	5	8.7%
	HO2	83	228	32.27					6.8%
	HO3	83	228	36.15					16.8%
Central	NA ^c	249	228	26.75	NA ^c	NA ^c	NA ^c	NA ^c	-1.8%

^a Although there are 248 test engines in the FD004 dataset, 20 engines were excluded as their data provided is insufficient for the LSTM model sequence length of 50.

^b IMP is short for improvement, which is defined as ('Local data only' RMSE – FedMA RMSE)/('Local data only' RMSE)

^c NA is short for not applicable.

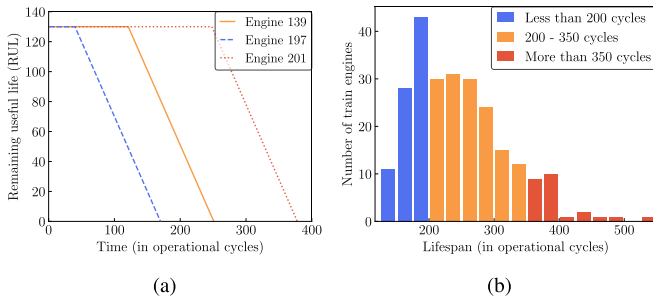


Fig. 9. (a) Piecewise RUL representation of selected train engines (b) distribution of train engine lifespans.

1) *Data Description*: The Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) turbofan engine degradation dataset [55] contains simulation results from run-to-failure experiments of multiple engines. The dataset is a collection of 4 sub-datasets, FD001 to FD004, which vary in the number of engines, fault types, and operating conditions studied. For each sub-dataset, predefined sets of train and test engines are also provided. Since FD004 is the most complex sub-dataset with engines experiencing 6 operating conditions and 2 fault types, it is selected for our FL experiment, which focuses on client data heterogeneity.

In the FD004 dataset, measurements for 21 sensor variables (e.g., pressure, temperature) are given for each operational time cycle of an engine. Following [10], sensors 1, 5, 6, 10, 16, 18, and 19 are not utilized for model training as they are either constant with time or exhibit erratic trends. The remaining sensors form the input features for the RUL estimation model. The RUL labels at each operational cycle of the train engines are not provided, but they can be constructed by taking the difference between the lifespan of the engine and the current operational cycle. Generally, it is more realistic to represent the progression of RUL as a non-linear, piecewise function because an engine's RUL remains fairly constant initially and only decreases after some time in operation when degradation begins. The empirical value of 130 [31], commonly used in turbofan engine literature, is used as the initial constant value for the RUL. The RUL labels of a few train engines are plotted in Fig. 9(a) to illustrate the piecewise representation. It can be seen that turbofan engines have non-cyclic and monotonous degradation patterns.

Next, the train engines are partitioned to form clients for FL training. Though the engines are of the same type, their lifespans can vary considerably depending on the operating conditions. Noting that the lifespans of train engines range from 128 to 543 operating cycles in Fig. 9(b), the train engines are divided into the following three categories:

- Short-life: Lifespan less than 200 cycles
- Moderate-life: Lifespan between 200 to 350 cycles
- Long-life: Lifespan more than 350 cycles

For the heterogeneous client formulation, a practical setting is envisaged, where clients have considerably different engine usage and data collection schedules. Thus, each client can end up with training data from only a particular category of lifespan distribution. These heterogeneous clients, HT1 to HT3, are detailed in the first three columns of Table VI. The clients differ in both the engine lifespan distribution as well as the engine dataset sizes owned. For instance, while client HT3 has data on longest lasting engines (more than 350 cycles), it also has the least amount of data (25 engines). To juxtapose the heterogeneous setting, a homogeneous partitioning of the client is also considered. The entire train engine dataset is randomly shuffled to form the clients, HO1 to HO3, as detailed in Table VI. Each client has engines with a lifespan distribution similar to the global train engine dataset and an equal amount of data (83 engines each).

2) *Local Client Model Formulation for RUL Estimation*: The FD004 test engines have the same feature variables as the train engines, but the data are only provided up to some random time before failure for each engine. Thus, the remaining operating cycles before failure, i.e., RUL, is not known. The problem setup is that the local clients are tasked with estimating the RUL of the test engines, which may have a different lifespan distribution from the client's available training data. Using each client's train engine data, an LSTM-based RUL estimation model relating the sensor data and the piecewise RUL labels is developed for the baseline client models. The best model architecture for the client models, which also forms the basis for the federated models, is determined in a manner broadly similar to Experiment 1. The sensor data are standardized first (zero mean and unit variance) and then fed into the LSTM model as smaller, uniform-length input sequences for model training. The model variations in Table IV were evaluated. All models were trained

for a maximum of 300 epochs with early stopping. A one-layer LSTM network with 256 neurons, encompassed by a fully connected input and output layer (therefore, $Q = 2$), emerged as the best-performing RUL estimation model when assessed on the test engines.

3) *Federated Model Development for RUL Estimation:* Using the same 2-layer architecture, the federated models are developed next using the FedAvg and FedMA algorithms. For the FedAvg federated model development, recall from Section IV-A(2) that the local model retraining within each communication round has to be performed incrementally for E epochs per round to curtail FL training divergence. Since about 300 epochs were required for local model convergence in the baseline scenario without FL, a set of small and mid-ranged values of $E \in \{5, 10, 15, 50, 75\}$ were considered as candidate epochs. The best FedAvg federated model performance was achieved with $E = 15$ for heterogeneous clients and $E = 10$ for homogeneous clients. The FedMA model development, in contrast, favors longer local model retraining within each communication round as explained earlier in Section IV-A(3). Accordingly, local model retraining was conducted for a maximum of 200 epochs within each communication round.

4) *RUL Estimation Results and Discussion:* The FedMA and FedAvg federated models were trained up to 15 communication rounds for both the heterogeneous and homogeneous client settings. For each round, the federated model accuracy is assessed through the $RMSE$ of the RUL predictions on the test engines. The results are comprehensively presented in Table VI and discussed henceforth. The top model performance (lowest $RMSE$ of 26.75) is achieved by the ‘Central’ model, which gives the benchmark RUL estimation performance for the ideal scenario of centrally accessing the entire train engine dataset for model training without privacy limitations. As the performance of the ‘Central’ model is comparable to the $RMSE$ values reported in existing literature [9], [31], we can be confident that it is a high-quality benchmark for assessing the relative performance of the baseline client models and federated models.

The performance of the baseline client models with no FL varies widely depending on the degree of client heterogeneity. While the model performance of homogeneous clients was fairly uniform (as evidenced by the narrow $RMSE$ range of 32.27 to 36.15), the heterogeneous clients had extremes, with very good models (HT2 with an $RMSE$ of 27.76) and very poor models (HT3 with an $RMSE$ of 44.87). Nonetheless, the federated models outperformed all the baseline client models, highlighting once again the crucial need for FL if high-accuracy machine health prognostics models are to be achieved under privacy constraints. Between the two FL methods, the FedMA federated model consistently topped FedAvg’s performance in both the homogeneous and heterogeneous settings, with its $RMSE$ in the heterogeneous setting being only 1.8% lower than the theoretical best performance established by the ‘Central’ model. According to [25], FedMA performs better with higher quality local models. Thus, it is entirely consistent that FedMA’s outperformance to FedAvg was more pronounced in the heterogeneous setting, which had a high-quality local model (HT2). Due to its similarity matching

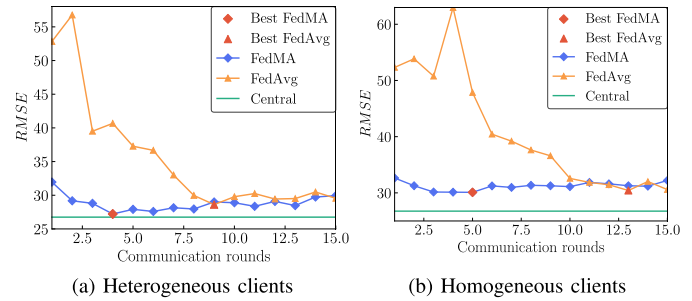


Fig. 10. Progression of the model accuracy for FedAvg and FedMA federated models under (a) heterogeneous and (b) homogeneous settings.

algorithm, FedMA is able to benefit from the presence of high-quality local models to a greater extent as the feature extractors from good local models are not diluted by naive parameter averaging. Fig. 10 plots the progression of the federated model accuracy for FedAvg and FedMA across the communication rounds. FedMA required less communication rounds for its best model, even in the homogeneous setting with only moderate quality local models, and converged to more optimal results than FedAvg.

In Section III, FedMA’s neuron matching behavior was only described mathematically, and its **practical interpretability** was not discussed. Thus, the feature extractors (neurons) of the local client models in the baseline scenario without FL, and after the first communication round of FedMA are visualized for the heterogeneous client setting in Fig. 11. Feature extractors are the LSTM layer’s output before it is fed to the fully connected dense layer. Only the first ten neurons of the 256 hidden neurons are plotted to allow for easy visual interpretation of the patterns. Starting with Fig. 11(a), which depicts the feature extractors of the local models in baseline setup without FL, it is clear that each client’s coordinate-wise neurons have learnt significantly different feature extraction traits from their heterogeneous training data. Hence, performing coordinate-wise model parameter averaging on dissimilar traits, as FedAvg does, will dampen the feature extraction capabilities of the federated model. Fig. 11(b) plots the feature extractors amidst local model training after the first communication round of FedMA, which has already matched and aligned similar neurons index-wise with just one communication round.

C. Complexity Analysis

In this section, the computational complexity of the feature matching-based health prognostic model is briefly discussed, together with some practical strategies for speed-ups. The computational complexity of an algorithm can refer to both space and time complexity [56].

In a FL setting, the clients are edge devices with limited computational resources [12] to perform offline model training and online prognostics, i.e., inferencing. Thus, space complexity is a greater concern for the client side than the central server. Space complexity, in terms of memory consumption, is determined by factors such as input data size, number of model parameters, and the type of optimization



Fig. 11. Visualization of feature extraction traits of local models in (a) the baseline setting without FL collaboration and after first round of (b) FedMA.

algorithms utilized [57]. For training the local LSTM-based client models, the complexity per weight and time step for updating model weights is $\mathcal{O}(1)$ [30] as the LSTM algorithm is local in time and space, i.e., each LSTM cell's output depends only on the previous cell's output and the current input. Furthermore, to reduce space complexity during training and inferencing, a light model architecture was chosen for the client model and the federated model, as mentioned in Section IV-A(2). For instance, our SoH estimation model has 68K parameters, which is a fraction of the 4.2M parameters of MobileNet [58], a popular parameter-efficient neural network for edge device deployment. Assuming parameters are stored in 32-bit floating-point numbers, the memory consumption of the model is approximately 2.7 Megabytes.

On the other hand, time complexity becomes the more critical factor in the federated model training phase as computing power is typically not a major constraint in the central server. The crux of the computational effort for the similarity-matched federated model development lies in the minimization of the matching cost matrix $\mathbf{P}$ via the Hungarian algorithm. If no hidden neurons are matched among clients, the worst case complexity per layer is $\mathcal{O}(D_{in} \cdot (JL)^2)$ for constructing $\mathbf{P}$ and $\mathcal{O}((JL)^3)$ for executing the Hungarian algorithm [25]. If all hidden neurons are matched, the best case complexity per layer is $\mathcal{O}(D_{in} \cdot L^2 + L^3)$ [25]. In practice, the complexity is likely to be better than the worst case [51] as indiscriminate growth of the federated model size is limited via the mechanism discussed in Section III-B(1). For practical implementation strategies, it is worth briefly noting that running time speed-up of the Hungarian algorithm can be achieved by using GPUs [59] and improved linear sum assignment solvers. More detailed information is available in [60] and [61].

Time complexity is also of interest during online prognostics by clients as inferences often need to be performed in near real-time. However, in comparison with offline federated

model training, the time complexity of online prognostics is lower as it only involves model inferencing using the well-trained federated model parameters and test data. Specifically, the inference process, which entails a forward pass of our one-layer LSTM-based federated model, has a complexity of approximately $\mathcal{O}(D_{in} \cdot L + L^2)$.

V. CONCLUSION

This paper argues that, in an increasingly privacy-concerned era, federated learning (FL) is critical for allowing local clients to learn from a diverse range of decentralized data and build high-quality industrial health prognostic models. Particularly, we proposed a feature matching-based FL model for SoH and RUL estimation of critical machinery, and demonstrated its aptness for building accurate federated models amidst heterogeneous degradation data obtained under complex practical conditions. The proposed approach outperformed both the baseline client models without FL and the standard Federated Averaging model due to its algorithm averaging only client neurons with similar feature extraction traits, which ensures that the helpful feature extractors of local client models are carried over undiluted to the resultant federated model. Amidst the nascency of FL studies for time series regression-based machine health modeling, our study is pivotal for spurring research into effective parameter aggregation methods for FL-based industrial health prognostics. Though the prognostic tasks studied in this paper use long short-term memory networks, the study can be extended to other deep neural networks in the future because the matched averaging is performed modularly layer-by-layer. Thus, as part of future works, the structurally simpler GRU network can be investigated for developing lighter health prognostics models in edge devices. Additionally, the technical feasibility of using an ensemble of privacy-preserving techniques such as differential privacy-enabled FL will also be studied for industrial health prognostics. Finally, as the current neuron matching algorithm allows for the nonparametric growth of the federated model and thus, susceptible to poisonous attacks, adversarial techniques to enhance the robustness and fault tolerance of the proposed model will be explored in future works.

REFERENCES

- [1] S. Munirathinam, "Industry 4.0: Industrial Internet of Things (IIOT)," in *Advances in Computers*, vol. 117, no. 1. Amsterdam, The Netherlands: Elsevier, 2020, pp. 129–164.
- [2] R. K. Mobley, *An Introduction to Predictive Maintenance*. Amsterdam, The Netherlands: Elsevier, 2002.
- [3] Y. Lei, N. Li, L. Guo, N. Li, T. Yan, and J. Lin, "Machinery health prognostics: A systematic review from data acquisition to RUL prediction," *Mech. Syst. Signal Process.*, vol. 104, pp. 799–834, May 2018.
- [4] S. Guo and W. Guo, "Process monitoring and fault prediction in multivariate time series using bag-of-words," *IEEE Trans. Autom. Sci. Eng.*, vol. 19, no. 1, pp. 230–242, Jan. 2022.
- [5] Q. Xu, M. Wu, E. Khoo, Z. Chen, and X. Li, "A hybrid ensemble deep learning approach for early prediction of battery remaining useful life," *IEEE/CAA J. Autom. Sinica*, vol. 10, no. 1, pp. 177–187, Jan. 2023.
- [6] Z. Pang, X. Si, C. Hu, and Z. Zhang, "An age-dependent and state-dependent adaptive prognostic approach for hidden nonlinear degrading system," *IEEE/CAA J. Autom. Sinica*, vol. 9, no. 5, pp. 907–921, May 2022.

- [7] S. Yao, Q. Kang, M. Zhou, M. J. Rawa, and A. Abusorrah, "A survey of transfer learning for machinery diagnostics and prognostics," *Artif. Intell. Rev.*, vol. 56, pp. 2871–2922, Aug. 2022.
- [8] L. D. Dong et al., "Modeling transmission rate of COVID-19 in regional countries to forecast newly infected cases in a nation by the deep learning method," in *Future Data and Security Engineering. Big Data, Security and Privacy, Smart City and Industry 4.0 Applications*. Singapore: Springer, Nov. 2021, pp. 411–423. [Online]. Available: https://link.springer.com/chapter/10.1007/978-981-16-8062-5_28#copyright-information
- [9] Z. Chen, M. Wu, R. Zhao, F. Guretno, R. Yan, and X. Li, "Machine remaining useful life prediction via an attention-based deep learning approach," *IEEE Trans. Ind. Electron.*, vol. 68, no. 3, pp. 2521–2531, Mar. 2021.
- [10] X. Zhang, Y. Qin, C. Yuen, L. Jayasinghe, and X. Liu, "Time-series regeneration with convolutional recurrent generative adversarial network for remaining useful life estimation," *IEEE Trans. Ind. Informat.*, vol. 17, no. 10, pp. 6820–6831, Oct. 2021.
- [11] J. Zhu, N. Chen, and W. Peng, "Estimation of bearing remaining useful life based on multiscale convolutional neural network," *IEEE Trans. Ind. Electron.*, vol. 66, no. 4, pp. 3208–3216, Apr. 2019.
- [12] D. C. Nguyen, M. Ding, P. N. Pathirana, A. Seneviratne, J. Li, and H. Vincent Poor, "Federated learning for Internet of Things: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 3, pp. 1622–1658, 3rd Quart., 2021.
- [13] C.-H. Lee and C.-H. Wu, "Learning to recognize driving patterns for collectively characterizing electric vehicle driving behaviors," *Int. J. Automot. Technol.*, vol. 20, no. 6, pp. 1263–1276, Dec. 2019.
- [14] A. Adadi, "A survey on data-efficient algorithms in big data era," *J. Big Data*, vol. 8, no. 1, p. 24, Jan. 2021.
- [15] L. Li, Y. Fan, M. Tse, and K.-Y. Lin, "A review of applications in federated learning," *Comput. Ind. Eng.*, vol. 149, Nov. 2020, Art. no. 106854.
- [16] R. Xu, N. Baracaldo, and J. Joshi, "Privacy-preserving machine learning: Methods, challenges and directions," 2021, *arXiv:2108.04417*.
- [17] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 2, pp. 1–19, 2019.
- [18] C. Dwork, "A firm foundation for private data analysis," *Commun. ACM*, vol. 54, no. 1, pp. 86–95, Jan. 2011.
- [19] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Found. Trends Theor. Comput. Sci.*, vol. 9, nos. 3–4, pp. 211–407, 2014.
- [20] R. L. Rivest, L. Adleman, and M. L. Dertouzos, "On data banks and privacy homomorphisms," *Found. Sec. Comput.*, vol. 4, no. 11, pp. 169–180, 1978.
- [21] J. Li, X. Kuang, S. Lin, X. Ma, and Y. Tang, "Privacy preservation for machine learning training and classification based on homomorphic encryption schemes," *Inf. Sci.*, vol. 526, pp. 166–179, Jul. 2020.
- [22] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, 2017, pp. 1273–1282.
- [23] B. McMahan, D. Ramage, K. Talwar, and L. Zhang, "Learning differentially private recurrent language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018, pp. 1–14. [Online]. Available: <https://openreview.net/pdf?id=BJ0hF1Z0b>
- [24] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," *Proc. Mach. Learn. Syst.*, vol. 2, pp. 429–450, Mar. 2020.
- [25] H. Wang, M. Yurochkin, Y. Sun, D. Papailiopoulos, and Y. Khazaeni, "Federated learning with matched averaging," 2020, *arXiv:2002.06440*.
- [26] J. Wang, Q. Liu, H. Liang, G. Joshi, and H. V. Poor, "Tackling the objective inconsistency problem in heterogeneous federated optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 7611–7623.
- [27] W. Zhang and X. Li, "Data privacy preserving federated transfer learning in machinery fault diagnostics using prior distributions," *Struct. Health Monit.*, vol. 21, no. 4, pp. 1329–1344, Jul. 2022.
- [28] Y. Liu et al., "Deep anomaly detection for time-series data in industrial IoT: A communication-efficient on-device federated learning approach," *IEEE Internet Things J.*, vol. 8, no. 8, pp. 6348–6358, Apr. 2021.
- [29] R. H. L. Rosero, C. Silva, and B. Ribeiro, "Remaining useful life estimation in aircraft components with federated learning," in *Proc. PHM Soc. Eur. Conf.*, 2020, vol. 5, no. 1, p. 9.
- [30] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [31] S. Zheng, K. Ristovski, A. Farahat, and C. Gupta, "Long short-term memory network for remaining useful life estimation," in *Proc. IEEE Int. Conf. Prognostics Health Manage. (ICPHM)*, Jun. 2017, pp. 88–95.
- [32] J. Zhang, P. Wang, R. Yan, and R. X. Gao, "Long short-term memory for machine remaining life prediction," *J. Manuf. Syst.*, vol. 48, pp. 78–86, Jul. 2018.
- [33] R. Zhao, J. Wang, R. Yan, and K. Mao, "Machine health monitoring with LSTM networks," in *Proc. 10th Int. Conf. Sens. Technol. (ICST)*, Nov. 2016, pp. 1–6.
- [34] Y. Choi, S. Ryu, K. Park, and H. Kim, "Machine learning-based lithium-ion battery capacity estimation exploiting multi-channel charging profiles," *IEEE Access*, vol. 7, pp. 75143–75152, 2019.
- [35] K. Park, Y. Choi, W. J. Choi, H. Ryu, and H. Kim, "LSTM-based battery remaining useful life prediction with multi-channel charging profiles," *IEEE Access*, vol. 8, pp. 20786–20798, 2020.
- [36] Y. Zhou, Y. Huang, J. Pang, and K. Wang, "Remaining useful life prediction for supercapacitor based on long short-term memory neural network," *J. Power Sources*, vol. 440, Nov. 2019, Art. no. 227149.
- [37] L. Ungurean, M. V. Micea, and G. Cârstoiu, "Online state of health prediction method for lithium-ion batteries, based on gated recurrent unit neural networks," *Int. J. Energy Res.*, vol. 44, no. 8, pp. 6767–6777, Jun. 2020.
- [38] P. Arora, R. E. White, and M. Doyle, "Capacity fade mechanisms and side reactions in lithium-ion batteries," *J. Electrochem. Soc.*, vol. 145, no. 10, p. 3647, 1998.
- [39] S. Dong, J. Xiao, X. Hu, N. Fang, L. Liu, and J. Yao, "Deep transfer learning based on bi-LSTM and attention for remaining useful life prediction of rolling bearing," *Rel. Eng. Syst. Saf.*, vol. 230, Feb. 2023, Art. no. 108914.
- [40] Y. Shen, B. Tang, B. Li, Q. Tan, and Y. Wu, "Remaining useful life prediction of rolling bearing based on multi-head attention embedded bi-LSTM network," *Measurement*, vol. 202, Oct. 2022, Art. no. 111803.
- [41] R. Guo, Y. Wang, H. Zhang, and G. Zhang, "Remaining useful life prediction for rolling bearings using EMD-RISI-LSTM," *IEEE Trans. Instrum. Meas.*, vol. 70, pp. 1–12, 2021.
- [42] Y. Qin, S. Adams, and C. Yuen, "Transfer learning-based state of charge estimation for lithium-ion battery at varying ambient temperatures," *IEEE Trans. Ind. Informat.*, vol. 17, no. 11, pp. 7304–7315, Nov. 2021.
- [43] E. Chemali, P. J. Kollmeyer, M. Preindl, R. Ahmed, and A. Emadi, "Long short-term memory networks for accurate state-of-charge estimation of Li-ion batteries," *IEEE Trans. Ind. Electron.*, vol. 65, no. 8, pp. 6730–6739, Aug. 2018.
- [44] Y. Qin, A. Arunan, and C. Yuen, "Digital twin for real-time Li-ion battery state of health estimation with partially discharged cycling data," *IEEE Trans. Ind. Informat.*, vol. 19, pp. 7247–7257, May 2023, doi: [10.1109/TII.2022.3230698](https://doi.org/10.1109/TII.2022.3230698).
- [45] K. Cho, B. van Merriënboer, D. Bahdanau, and Y. Bengio, "On the properties of neural machine translation: Encoder–decoder approaches," in *Proc. 8th Workshop Syntax, Semantics Struct. Stat. Transl. (SSST)*. Doha, Qatar: Association for Computational Linguistics, Oct. 2014, pp. 103–111.
- [46] R. Cahuantzi, X. Chen, and S. Güttel, "A comparison of LSTM and GRU networks for learning symbolic sequences," 2021, *arXiv:2107.02248*.
- [47] K. Irie, Z. Tüske, T. Alkhoul, R. Schlüter, and H. Ney, "LSTM, GRU, highway and a bit of attention: An empirical overview for language modeling in speech recognition," in *Proc. Interspeech*, Sep. 2016, pp. 3519–3523.
- [48] N. Gruber and A. Jockisch, "Are GRU cells more specific and LSTM cells more sensitive in motive classification of text?" *Frontiers Artif. Intell.*, vol. 3, p. 40, Jun. 2020.
- [49] Y. Qin, C. Yuen, Y. Shao, B. Qin, and X. Li, "Slow-varying dynamics-assisted temporal capsule network for machinery remaining useful life estimation," *IEEE Trans. Cybern.*, vol. 53, no. 1, pp. 592–606, Jan. 2023, doi: [10.1109/TCYB.2022.3164683](https://doi.org/10.1109/TCYB.2022.3164683).
- [50] Z. Ghahramani and T. Griffiths, "Infinite latent feature models and the Indian buffet process," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 18, 2005, pp. 1–8.
- [51] M. Yurochkin, M. Agarwal, S. Ghosh, K. Greenewald, N. Hoang, and Y. Khazaeni, "Bayesian nonparametric federated learning of neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 7252–7261.
- [52] H. W. Kuhn, "The Hungarian method for the assignment problem," *Nav. Res. Logistics Quart.*, vol. 2, nos. 1–2, pp. 83–97, Mar. 1955.
- [53] B. Saha and K. Goebel, "Battery data set," NASA Ames Prognostics Data Repository, NASA Ames Res. Center, Moffett Field, CA, USA, 2007. [Online]. Available: <http://ti.arc.nasa.gov/project/prognostic-data-repository>

- [54] Q. Ma, Y. Zheng, W. Yang, Y. Zhang, and H. Zhang, "Remaining useful life prediction of lithium battery based on capacity regeneration point detection," *Energy*, vol. 234, Nov. 2021, Art. no. 121233.
- [55] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in *Proc. Int. Conf. Prognostics Health Manage.*, Oct. 2008, pp. 1–9.
- [56] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*. Cambridge, MA, USA: MIT Press, 2022.
- [57] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [58] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," *CoRR*, vol. abs/1704.04861, pp. 1–9, Apr. 2017.
- [59] K. Date and R. Nagi, "GPU-accelerated Hungarian algorithms for the linear assignment problem," *Parallel Comput.*, vol. 57, pp. 52–72, Sep. 2016.
- [60] *Lapsolver 1.1.0*. Accessed: Mar. 21, 2023. [Online]. Available: <https://pypi.org/project/lapsolver/>
- [61] C. Heindl. (2022). *Py-Lapsolver*. [Online]. Available: <https://github.com/cheind/py-lapsolver>



Anushiya Arunan received the B.Eng. degree in environmental engineering and the B.Soc.Sci. degree in economics from the National University of Singapore, Singapore, in 2014. She is currently pursuing the Ph.D. degree with the Singapore University of Technology and Design, Singapore. Her research interests include time series analytics, federated learning, data-driven process monitoring, and machine health prognostics.



Yan Qin (Member, IEEE) received the B.S. degree in electronic information engineering from Information Engineering University, Zhengzhou, China, in 2011, the M.S. degree in control theory and control engineering from Northeastern University, Shenyang, China, in 2013, and the Ph.D. degree in control science and engineering from Zhejiang University, Hangzhou, China, in 2018. From 2019 to 2022, he was a Post-Doctoral Research Fellow with the Singapore University of Technology and Design and Nanyang Technological University. Since 2023, he has been an Associate Professor with the School of Automation, Chongqing University, China. His research interests include data-driven process monitoring, soft sensor modeling, and remaining useful life estimation for industrial processes and essential assets.



Xiaoli Li (Senior Member, IEEE) is currently the Department Head and a Principal Scientist of the Machine Intelligence (MI) Department, Institute for Infocomm Research (I2R), A*STAR, Singapore. He is also an Adjunct Full Professor with the School of Computer Science and Engineering, Nanyang Technological University. He serves as an AI Advisor for the Smart Nation and Digital Government Office (SNDGO), Prime Minister's Office Singapore. His research interests include artificial intelligence (AI), data mining, machine learning, and bioinformatics, with over 300 peer-reviewed papers published in top conferences and journals in these fields. He has won nine best paper awards for his work, which has been highly influential in advancing the state-of-the-art in these fields. He is also highly respected in the academic community for his service as the Editor-in-Chief of the *Annual Review of Artificial Intelligence*, an Associate Editor of *IEEE TRANSACTIONS ON ARTIFICIAL INTELLIGENCE* and *Knowledge and Information Systems*, and conference chairs/area chairs in leading AI and data analytics conferences. He has been identified as one of the world's top 2% scientists in the AI domain by Stanford University.



Chau Yuen (Fellow, IEEE) received the B.Eng. and Ph.D. degrees from Nanyang Technological University, Singapore, in 2000 and 2004, respectively.

He was a Post-Doctoral Fellow with Lucent Technologies Bell Labs, Murray Hill, NJ, USA, in 2005, and a Visiting Assistant Professor with The Hong Kong Polytechnic University in 2008. From 2006 to 2010, he was with the Institute for Infocomm Research, Singapore. From 2010 to 2023, he was with the Engineering Product Development Pillar, Singapore University of Technology and Design. Since 2023, he has been with the School of Electrical and Electronic Engineering, Nanyang Technological University. He has published over 500 research papers in international journals or conferences. He holds three U.S. patents.

Dr. Yuen was a recipient of the Lee Kuan Yew Gold Medal, the Institution of Electrical Engineers Book Prize, the Institute of Engineering of Singapore Gold Medal, the Merck Sharp and Dohme Gold Medal, and twice a recipient of the Hewlett Packard Prize. He received the IEEE Asia Pacific Outstanding Young Researcher Award in 2012 and the IEEE VTS Singapore Chapter Outstanding Service Award in 2019. He received the IEEE Communications Society Fred W. Ellersick Prize in 2023, the IEEE Marconi Prize Paper Award in Wireless Communications in 2021, and the EURASIP Best Paper Award for *Journal on Wireless Communications and Networking* in 2021. He serves as the Editor-in-Chief for *Springer Nature Computer Science* and an Editor for *IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY*, *IEEE SYSTEMS JOURNAL*, and *IEEE TRANSACTIONS ON NETWORK SCIENCE AND ENGINEERING*, where he was awarded as the IEEE TRANSACTIONS ON NETWORK SCIENCE AND ENGINEERING Excellent Editor Award and a Top Associate Editor for *IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY* from 2009 to 2015. He also served as the Guest Editor for several special issues, including *IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS*, *IEEE Wireless Communications Magazine*, *IEEE Communications Magazine*, *IEEE Vehicular Technology Magazine*, *IEEE TRANSACTIONS ON COGNITIVE COMMUNICATIONS AND NETWORKING*, and *Applied Energy* (Elsevier). He is a Distinguished Lecturer of the IEEE Vehicular Technology Society, a Top 2% Scientist by Stanford University, and a Highly Cited Researcher by Clarivate Web of Science.